P0009r00 : Polymorphic Multidimensional Array View

Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Christian Trott
Contact: crtrott@sandia.gov
Author: Juan Alday
Contact: juanalday@gmail.com
Author: Jesse Perla
Contact: jesse.perla@ubc.ca
Author: Mauro Bianco
Contact: mbianco@cscs.ch
Author: Robin Maffeo
Contact: Robin.Maffeo@amd.com
Author: Ben Sander
Contact: ben.sander@amd.com
Author: Bryce Lelbach
Contact: balelbach@lbl.gov

Number: P0009
Version: 00
Date: 2015-09-23
URL: https://github.com/kokkos/PolyView/blob/master/P0009.rst
WG21: Library Evolution Working Group (LEWG)
WG21: Evolution Working Group (EWG) for array declaration

1 Rationale for polymorphic multidimensional array view

Multidimensional arrays are a foundational data structure for science and engineering codes, as demonstrated by their extensive use in FORTRAN for five decades. A multidimensional array is a **view** to a memory extent through a **layout** mapping from a multi-index space (domain) to that extent (range). A view's layout mapping may be bijective as in the case of a traditional multidimensional array, injective as in the case of a subarray, or surjective to express symmetry.

Traditional layout mappings have been specfied as part of the language. For example, FORTRAN specifies *column major* layout and C specifies *row major* layout. Such a language-imposed specification requires signficant code refactoring to change an array's layout, and requires significant code complexity to implement non-traditional layouts such as tiling in modern linear algebra or structured grid application domains. Such layout changes are required to adapt and optimize code for varying computer architectures; for example, to change a code from *array of structures* to *structure of arrays*. Furthermore, multiple versions of code must be maintained for each required layout.

A multidimensional array view abstraction with polymorphic layout is required to enable changing array layout without extensive code refactoring and maintenance of functionally redundant code. Layout polymorphism is a critical capability; however, it is not the only beneficial form of polymorphism.

The Kokkos library (github.com/kokkos/kokkos) implements multidimensional array views with polymorphic layout, and other access properties as well. Until recently the Kokkos implementation was limited to C++1998 standard and is incrementally being refactored to C++2011 standard. As such deviations from the Kokkos interface that leverage C++2011 capabilities which are included in this proposed have been successfully prototyped in a separate repository (github.com/kokkos/PolyView).

2 Extensibility beyond multdimensional array layout

The polymorphic **view** abstraction and interface has utility well beyond the multidimensional array layout property. Other planned and prototyped properties include specification of which *memory space* within a heterogeneous memory system the viewed data resides and algorithmic access intent properties. Examples of access intent properties include (1) *read-only random with locality* such that member queries are performed through GPU texture cache hardware for GPU memory spaces, (2) *atomic* such that member access operations are overloaded via proxy objects to atomic operations (see P0019, Atomic View), (3) *non-temporal* such that member access operations can be overloaded with non-caching reads and writes, and (4) *restrict* to guarantee

non-aliasing of viewed data within the current context.

3 Compare and contrast summary re: previous array_view proposal

The essential issue with **array_view** in N4512 "Multidimensional bounds, offset and array_view, revision 7" is that it did not fulfill C++'s *zero-overhead abstraction* requirement (for both dynamic and static extents), and does not provide a zero-overhead abstraction to different memory layouts which are essential for library interoperability with a variety of C++ (e.g. Eigen) and other languages (e.g. Fortran and Matlab's C++ interface). Were it to be accepted, another library would be necessary to provide "direct mapping to the hardware" for views of arrays. Many of the issues are discussed in more detail in N4355, N4300, and N4222.

Unlike N4512, this proposal

- allows the layout to be more general with different orderings and padding essential for performant member access (*e.g.*, caching, coalescing),
- enables interoperability with libraries using compile-time extents, and
- achieves zero-overhead abstraction for *constexpr* extents and strides which provides more opportunities for the compiler to optimize array data member access functions.

Furthemore, this proposal provides a path for seemless extensibility for view properties beyond array dimensions and array layout. Thus we choose the name **view** as opposed to **array_view** in anticipation of this new fundamental mechanism for *viewing* spans of memory with an easily extensible set of properties.

4 The View

The proposed **view** has template arguments for the data type of the array and a parameter pack for polymorphic properties of the view.

```
namespace std {
namespace experimental {
    template< class DataType , class ... Properties >
    struct view ;
}}
```

The complete proposed specification for **view** is included at the end of this paper. We present the specification incrementally to convey the rational for this specification.

An initial set of view properties are proposed. These properties are defined by class and templated class types. It is an open question as to whether these view properties should resided in the same namespace as the **view**, or in a designated namespace such as the **std::rel_ops** functions, **std::chrono** classes, or **std::regex_constants** constants.

```
namespace std {
namespace experimental {
namespace view_property {
    // view property classes
}}}
```

5 View of a One-Dimensional Array

A view of a one-dimension array is anticipated to subsume the functionality of a pointer to memory extent combined with an array length. For example, a one-dimensional array is passed to a function as follows.

```
void foo( int array[] , size_t N ); // Traditional API
void foo( const int array[] , size_t N ); // Traditional API

void foo( view< int[] > array ); // View API
void foo( view< const int[] > array ); // View API

void bar()
{
  enum { L = ... };
  int buffer[ L ];
  view<int[]> array( buffer , L );
```

```
  assert( L == array.size() );
  assert( & array[0] == buffer );

  foo( array );
}
```

The *const-ness* of a view is analogous to the *const-ness* of a pointer. A const-view is similar to a const-pointer in that the view may not be modifid but the viewed extent of memory may be modified. A view-of-const is similar to a pointer-to-const in that the viewed extent of memory may not be modified.

6 View of Traditional Multidimensional Array with Explicit Dimensions

A traditional multidimensional array with explicit dimensions (for example, an array of 3x3 tensors) is passed to a function as follows.

```
void foo( double array[][3][3] , size_t N0 ); // Traditional API
void foo( view< double[][3][3] > array ); // View API

void bar()
{
  enum { L = ... };
  int buffer[ L * 3 * 3 ];
  view< double[][3][3] > array( buffer , L );

  assert( 3 == array.rank() );
  assert( L == array.extent(0) );
  assert( 3 == array.extent(1) );
  assert( 3 == array.extent(2) );
  assert( array.size() == array.extent(0) * array.extent(1) * array.extent(2) );
  assert( & array(0,0,0) == buffer );

  foo( array );
}
```

7 View of Multidimensional Array with Multiple Implicit Dimensions (Preferred)

Requires slight language specification change for correction and relaxation of array declaration.

Multidimensional arrays are used with multiple implicit dimensions; i.e., more dimensions than the leading dimension are declared at runtime. Such arrays are implemented within applications and libraries with numerous design idioms.

A minimalist design that preserves the appearance of conventional multidimensional array syntax follows an *array of pointers to array of pointers to ...* idiom. While dereferencing operations are syntactically compatible with an array of explicitly declared dimensions this idiom provides no locality guarantees for members of the array, consumes significant memory for the arrays of pointers, and is problematic when passing such arrays to functions.

```
double *** x ;
x = new double **[N0];
for ( size_t i0 = 0 ; i0 < N0 ; ++i0 ) {
  x[i0] = new double *[N1];
  for ( size_t i1 = 0 ; i1 < N1 ; ++i1 ) {
    x[i0][i1] = new double[N2] ;
  }
}

x[i0][i1][i2] // member access

foo( double *const *const * const array , size_t N0 , size_t N1 , size_t N2 );
```

A major goal of the **view** interface is to preserve compatibility between views to arrays with explicit and implicitly declared dimensions. In the following example foo1 and foo2 accept rank 3 arrays of integers with prescribed explicit / implicit dimensions and fooT accepts a rank 3 array of integers with unprescribed dimensions.

```
void foo1( view< int[ ][3][3] > array ); // Two explicit dimensions
void foo2( view< int[ ][ ][ ] > array ); // All implicit dimensions
```

```
// Accept a view of a rank three array with value type int
// and dimensions are explicit or implicit.
template< class T , class ... P >
typename std::enable_if< view<T,P...>::rank() == 3 >::type
foo( view<T,P...> array );

void bar()
{
  enum { L = ... };
  int buffer[ L * 3 * 3 ];
  view< int[][][] > array( buffer , L , 3 , 3 );

  assert( 3 == array.rank() );
  assert( L == array.extent(0) );
  assert( 3 == array.extent(1) );
  assert( 3 == array.extent(2) );
  assert( array.size() == array.extent(0) * array.extent(1) * array.extent(2) );
  assert( & array(0,0,0) == buffer );

  foo( array );
}
```

7.1 Relaxed array type declarator

The current array type declarator constraints are defined in in **8.3.4 Arrays paragraph 3** as follows.

> *When several "array of" specifications are adjacent, a multidimensional array is created; only the first of the constant expressions that specify the bounds of the arrays may be omitted.*

Note that this existing specification is in error when array syntax is used in a type definition - a type definition does not create a multidimensional array.

```
typedef int X[][3][3] ; // does not create a multidimensional array
using Y = int[][3][3] ; // does not create a multidimensional array
```

This syntax requires a relaxation of array type declarator constraints defined in **8.3.4 Arrays paragraph 3**.

Relaxing the **8.3.4.p3** constraint as follows will

- clarify the difference between array type and array object declarations,
- preserve correctness for conventional array object declarations,
- allow the proposed syntax for a view of an array with multiple implicit dimensions, and
- have zero impact on a compiler's processing of executable statements.

> *When several "array of" specifications are adjacent to form a multidimensional array type specification and that type is used in the explicit declaration of a multidimensional array then only the first of the sequence of array bound constant expressions may be omitted; otherwise any or all of the array bound constant expressions may be omitted.*

There exists at least two precedents for types that can be defined but not used to declare objects: (1) an array with an omitted leading bound and (2) **void**.

Relaxing this constraint is a simple one-line change in Clang that merely disables the error message and allows omission of second and subsequent dimensions.

In gcc 4.7, 4.8, and 4.9 this relaxation was implicitly supported as demonstrated by the following error-free and warning-free meta function.

```
template< typename T , unsigned R >
struct implicit_array_type { using type = typename implicit_array_type<T,R-1>::type[] ; };

template< typename T >
struct implicit_array_type<T,0> { using type = T ; };

using array_rank_3 = typename implicit_array_type<int,3>::type ;
```

7.2 Simplification for array with all implicit dimensions

A *properties* mechanism is defined to simplify declaration of a view to array of rank **R** with all-implicit dimensions.

```
template< typename T , unsigned R >
using array_view = view< T , view_property::implicit_dimensions< R > > ;
```

```
// Equivalent types:
array_view<int,3>
view<int[][][]>
```

8 View of Multidimensional Array with Multiple Implicit Dimensions (alternative)

If the array declaration constraint in **8.3.4.p3** is not relaxed then an alternative mechanism will be required to define mixed explicit and implicit dimensions through a view dimension property. A dimension property is syntactically more verbose and requires the "magic value" zero to denote an implicit dimension. The "magic value" of zero is chosen for consistency with **std::extent**.

```
view< int[][][3] > x(ptr,N0,N1); // preferred concise syntax
view< int , view_property::dimension<0,0,3> > y(ptr,N0,N1); // verbose syntax

assert( extent< int[][][3] , 0 >::value == 0 );
assert( extent< int[][][3] , 1 >::value == 0 );
assert( extent< int[][][3] , 2 >::value == 3 );

assert( view_property::dimension<0,0,3>::extent_0 == 0 );
assert( view_property::dimension<0,0,3>::extent_1 == 0 );
assert( view_property::dimension<0,0,3>::extent_2 == 3 );

assert( x.extent(0) == N0 );
assert( x.extent(1) == N1 );
assert( x.extent(2) == 3 );

assert( y.extent(0) == N0 );
assert( y.extent(1) == N1 );
assert( y.extent(2) == 3 );
```

If this alternative *properties* mechanism is required then the simple array declaration syntax is still available and will be supported when only the leading dimension is implicit.

```
view< int[] > x ; // concise syntax
view< int , view_property::dimension<0> > y ; // property syntax
```

A concern with this alternative *properties* mechanism is that if a zero value becomes accepted within dimension statements then there is potential confusion between implicit dimensions and explicit dimensions of zero. For example, are the following declarations equivalent?

```
view<int[0][0]> // If permitted
view<int, view_property::dimension<0,0> >
```

9 View Properties: Layout Polymorphism

The **view::operator()** maps the input multi-index from the array's cartesian product multi-index *domain* space to a member in the array's *range* space. This is the **layout** mapping for the viewed array. For natively declared multidimensional arrays the layout mapping is defined to conform to treating the multidimensional array as an *array of arrays of arrays ...*; i.e., the size and span are equal and the strides increase from right-to-left. In the FORTRAN language defines layout mapping with strides increasing from left-to-right. These *native* layout mappings are only two of many possible layouts. For example, the *basic linear algebra subprograms (BLAS)* standard defines dense matrix layout mapping with padding of the leading dimension, requiring both dimensions and **LDA** parameters to fully declare a matrix layout.

A view property template parameter specifies a layout mapping. If this property is omitted the layout mapping of the view conforms to a corresponding natively declared multidimensional array as if implicit dimensions were declared explicitly. The default layout is *regular* - the distance is constant between entries when a single index of the multi-index is incremented. This distance is the *stride* of the corresponding dimension. The default layout mapping is bijective and the stride increases monotonically from the right most to the left most dimension.

```
// The default layout mapping of a rank-four multidimensional
// array is as if implemented as follows.

template< size_t N0 , size_t N1 , size_t N2 , size_t N3 >
size_t native_mapping( size_t i0 , size_t i1 , size_t i2 , size_t i3 )
  {
    return i0 * N3 * N2 * N1 // stride == N3 * N2 * N1
         + i1 * N3 * N2      // stride == N3 * N2
         + i2 * N3           // stride == N3
```

```
        + i3 ;              // stride == 1
  }
```

An initial set of layout properties are **layout_right**, **layout_left**, and **layout_stride**.

```
  namespace std {
  namespace experimental {
  namespace view_property {
      struct layout_right ;
      struct layout_left ;
      struct layout_stride ;
  }}}
```

```
typedef view< int[][][] > view_native ;
typedef view< int[][][] , view_property::layout_right > view_right ;
typedef view< int[][][] , view_property::layout_left >  view_left ;

assert( std::is_same< typename view_native::layout , void >::value );
assert( std::is_same< typename view_right ::layout , view_property::layout_right >::value );
assert( std::is_same< typename view_left  ::layout , view_property::layout_left >::value );

assert( view_native::is_regular::value );
assert( view_right ::is_regular::value );
assert( view_left  ::is_regular::value );
```

A **void** (*a.k.a.*, default or native) mapping is regular and bijective with strides increasing from increasing from right most to left most dimension. A **layout_right** mapping is regular and injective (may have padding) with strides increasing from right most to left most dimension. A **layout_left** mapping is regular and injective (may have padding) with strides increasing from left most to right most dimension. A **layout_stride** mapping is regular; however, it may not be injective or surjective.

```
// The right and left layout mapping of a rank-four
// multidimensional array could be is as if implemented
// as follows.  Note that padding is allowed but not required.

template< size_t N0 , size_t N1 , size_t N2 , size_t N4 >
size_t right_mapping( size_t i0 , size_t i1 , size_t i2 , size_t i3 )
  {
    const size_t S3 = // stride of dimension 3
    const size_t P3 = // padding of dimension 3
    const size_t P2 = // padding of dimension 2
    const size_t P1 = // padding of dimension 1
    return i0 * S3 * ( P3 + N3 ) * ( P2 + N2 ) * ( P1 + N1 )
         + i1 * S3 * ( P3 + N3 ) * ( P2 + N2 )
         + i2 * S3 * ( P3 + N3 )
         + i3 * S3 ;
  }

template< size_t N0 , size_t N1 , size_t N2 , size_t N4 >
size_t left_mapping( size_t i0 , size_t i1 , size_t i2 , size_t i3 )
  {
    const size_t S0 = // stride of dimension 0
    const size_t P0 = // padding of dimension 0
    const size_t P1 = // padding of dimension 1
    const size_t P2 = // padding of dimension 2
    return i0 * S0
         + i1 * S0 * ( P0 + N0 )
         + i2 * S0 * ( P0 + N0 ) * ( P1 + N1 )
         + i3 * S0 * ( P0 + N0 ) * ( P1 + N1 ) * ( P2 + N2 );
  }
```

10 View Properties: Extensible Layout Polymorphism

The **view** is intended to be extensible such that a user may supply a customized layout mapping. A user supplied customized layout mapping will be required to conform to a specified interface; *a.k.a.*, a C++ Concept. Details of this extension point will be included in a subsequent proposal.

An important customized layout mapping is hierarchical tiling. This kind of layout mapping is used in dense linear algebra matrices and computations on Cartesian grids to improve the spatial locality of array entries. These mappings are bijective but are not regular. Computations on such multidimensional arrays typically iterate through tiles as *subviews* of the array.

```
template< size_t N0 , size_t N1 , size_t N2 >
size_t tiling_left_mapping( size_t i0 , size_t i1 , size_t i2 )
{
```

```
  static constexpr size_t T = // cube tile size
  constexpr size_t T0 = ( N0 + T - 1 ) / T ; // tiles in dimension 0
  constexpr size_t T1 = ( N1 + T - 1 ) / T ; // tiles in dimension 1
  constexpr size_t T2 = ( N2 + T - 1 ) / T ; // tiles in dimension 2

  // offset within tile + offset to tile
  return ( i0 % T ) + T * ( i1 % T ) + T * T * ( i2 % T )
       + T * T * T * ( ( i0 / T ) + T0 * ( ( i1 / T ) + T1 * ( i2 / T ) ) );
}
```

Note that a tiled layout mapping is irregular and if padding is required to align with tile boundarries then the span will exceed the size. A customized layout mapping will have slightly different requirements depending on whether the layout is regular or irregular.

11 View Properties: Flexibility and Extensibility

One or more view properties of **void** are acceptable and have no effect. This allows user code to define a template argument list of potential view properties and then enabling/disabling a particular property by conditionally setting it to **void**. For example:

```
typedef typename std::conditional< AllowPadding , view_property::layout_right , void >::type  layout ;

// If AllowPadding then use layout_right else use native layout
typedef view< int[][][] , layout > MyViewType ;
```

12 Specification with Simple View Properties

Simple view properties include the array layout and if necessary a **view_property::dimension** type for arrays with multiple implicit dimensions. View properties are provided through a variadic template to support extensibility of the view. Possible additional properties include array bounds checking, atomic access to members, memory space within a heterogeneous memory architecture, and user access pattern hints.

```
namespace std {
namespace experimental {

template< class DataType , class ... Properties >
struct view {
  //--------------------
  // Types:

  // Types are implementation and Properties dependent.
  // The following type implementation are normative
  // with respect to empty Properties.

  using value_type = typename std::remove_all_extents< DataType >::type ;
  using reference = value_type & ; // Typical type, but implementation defined
  using pointer = value_type * ; // Typical type, but implementation defined

  //--------------------
  // Domain index space properties:

  static constexpr unsigned rank() const ;

  template< typename IntegralType >
  constexpr size_t extent( const IntegralType & ) const ;

  // Cardinality of index space; i.e., product of extents
  constexpr size_t size() const ;

  //--------------------
  // Layout mapping properties:

  using layout = array layout type ;
  using is_regular = std::integral_constant<bool, B > ;
```

```
// If the layout mapping is regular then return the
// distance between members when index # is increased by one.
template< typename IntegralType >
constexpr size_t stride( const IntegralType & ) const ;

// Span covering the members
constexpr size_t span() const ;

// Span of an array with regular layout if it
// is constructed with the given implicit dimensions.
static constexpr
    size_t span( size_t implicit_N0
        , size_t implicit_N1 = 0
        , size_t implicit_N2 = 0
        , size_t implicit_N3 = 0
        , size_t implicit_N4 = 0
        , size_t implicit_N5 = 0
        , size_t implicit_N6 = 0
        , size_t implicit_N7 = 0
        , size_t implicit_N8 = 0
        , size_t implicit_N9 = 0
        );

// Pointer to member memory
constexpr pointer data() const ;

//--------------------
// Member access (proper):

// EnableIf rank == 0
reference operator()() const ;

// EnableIf rank == 1 and std::is_integral<t0>::value
template< typename t0 >
reference operator[]( const t0 & i0 ) const ;

// EnableIf rank == 1 and std::is_integral<t0>::value
template< typename t0 >
reference operator()( const t0 & i0 ) const ;

// EnableIf rank == 2 and std::is_integral<t#>::value
template< typename t0 , typename t1 >
reference operator()( const t0 & i0
    , const t1 & i1 ) const ;

// EnableIf rank == 3 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const t2 & i2 ) const ;

// EnableIf rank == 4 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 , typename t3 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const t2 & i2
    , const t3 & i3
    ) const ;

// EnableIf rank == 5 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 >
reference operator()( const t0 & i0
```

```
    , const t1 & i1
    , const t2 & i2
    , const t3 & i3
    , const t4 & i4
    ) const ;

// EnableIf rank == 6 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 , typename t5 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const t2 & i2
    , const t3 & i3
    , const t4 & i4
    , const t5 & i5
    ) const ;

// EnableIf rank == 7 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 , typename t5 , typename t6 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const t2 & i2
    , const t3 & i3
    , const t4 & i4
    , const t5 & i5
    , const t6 & i6
    ) const ;

// EnableIf rank == 8 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 , typename t5 , typename t6 ,
typename t7 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const t2 & i2
    , const t3 & i3
    , const t4 & i4
    , const t5 & i5
    , const t6 & i6
    , const t7 & i7
    ) const ;

// EnableIf rank == 9 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 , typename t5 , typename t6 ,
typename t7 , typename t8 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const t2 & i2
    , const t3 & i3
    , const t4 & i4
    , const t5 & i5
    , const t6 & i6
    , const t7 & i7
    , const t8 & i8
    ) const ;

// EnableIf rank == 10 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 , typename t5 , typename t6 ,
typename t7 , typename t8 , typename t9 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const t2 & i2
    , const t3 & i3
    , const t4 & i4
```

```
    , const t5 & i5
    , const t6 & i6
    , const t7 & i7
    , const t8 & i8
    , const t9 & i9
    ) const ;

//--------------------
// Member access (improper):

// EnableIf rank == 0 and i# == 0
reference operator()( const int i0
    , const int i1 = 0
    , const int i2 = 0
    , const int i3 = 0
    , const int i4 = 0
    , const int i5 = 0
    , const int i6 = 0
    , const int i7 = 0
    , const int i8 = 0
    , const int i9 = 0
    ) const ;

// EnableIf rank == 1 and std::is_integral<t0>::value and i{1-9} == 0
template< typename t0 >
reference operator()( const t0 & i0
    , const int i1
    , const int i2 = 0
    , const int i3 = 0
    , const int i4 = 0
    , const int i5 = 0
    , const int i6 = 0
    , const int i7 = 0
    , const int i8 = 0
    , const int i9 = 0
    ) const ;

// EnableIf rank == 2 and std::is_integral<t#>::value
template< typename t0 , typename t1 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const int i2
    , const int i3 = 0
    , const int i4 = 0
    , const int i5 = 0
    , const int i6 = 0
    , const int i7 = 0
    , const int i8 = 0
    , const int i9 = 0
    ) const ;

// EnableIf rank == 3 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const t2 & i2
    , const int i3
    , const int i4 = 0
    , const int i5 = 0
    , const int i6 = 0
    , const int i7 = 0
    , const int i8 = 0
```

```
    , const int i9 = 0
    ) const ;

// EnableIf rank == 4 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 , typename t3 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const t2 & i2
    , const t3 & i3
    , const int i4
    , const int i5 = 0
    , const int i6 = 0
    , const int i7 = 0
    , const int i8 = 0
    , const int i9 = 0
    ) const ;

// EnableIf rank == 5 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const t2 & i2
    , const t3 & i3
    , const t4 & i4
    , const int i5
    , const int i6 = 0
    , const int i7 = 0
    , const int i8 = 0
    , const int i9 = 0
    ) const ;

// EnableIf rank == 6 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 , typename t5 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const t2 & i2
    , const t3 & i3
    , const t4 & i4
    , const t5 & i5
    , const int i6
    , const int i7 = 0
    , const int i8 = 0
    , const int i9 = 0
    ) const ;

// EnableIf rank == 7 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 , typename t5 , typename t6 >
reference operator()( const t0 & i0
    , const t1 & i1
    , const t2 & i2
    , const t3 & i3
    , const t4 & i4
    , const t5 & i5
    , const t6 & i6
    , const int i7
    , const int i8 = 0
    , const int i9 = 0
    ) const ;

// EnableIf rank == 8 and std::is_integral<t#>::value
template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 , typename t5 , typename t6 ,
typename t7 >
```

```
        reference operator()( const t0 & i0
            , const t1 & i1
            , const t2 & i2
            , const t3 & i3
            , const t4 & i4
            , const t5 & i5
            , const t6 & i6
            , const t7 & i7
            , const int i8
            , const int i9 = 0
            ) const ;

        // EnableIf rank == 9 and std::is_integral<t#>::value
        template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 , typename t5 , typename t6 ,
        typename t7 , typename t8 >
        reference operator()( const t0 & i0
            , const t1 & i1
            , const t2 & i2
            , const t3 & i3
            , const t4 & i4
            , const t5 & i5
            , const t6 & i6
            , const t7 & i7
            , const t8 & i8
            , const int i9
            ) const ;

        //--------------------
        // Construct/copy/destroy:

        ~view();
        constexpr view();
        constexpr view( const view & );
        constexpr view( view && );
        view & operator = ( const view & );
        view & operator = ( view && );

        constexpr view( pointer
            , size_t implicit_N0 = 0
            , size_t implicit_N1 = 0
            , size_t implicit_N2 = 0
            , size_t implicit_N3 = 0
            , size_t implicit_N4 = 0
            , size_t implicit_N5 = 0
            , size_t implicit_N6 = 0
            , size_t implicit_N7 = 0
            , size_t implicit_N8 = 0
            , size_t implicit_N9 = 0
            );

        template< class UType , class ... UProperties >
        constexpr view( const view< UType , UProperties ... > & );

        template< class UType , class ... UProperties >
        view & operator = ( const view< UType , UProperties ... > & );
    };

    }}
```

using value_type = typename std::remove_all_extents< DataType >::type ;

using reference =

The type returned by a dereferencing operator. Typically this will be **value_type &**. [Note: The reference type may be a proxy depending upon the **Properties**. For example, if a property indicates that all member references are to be atomic then the reference type would be a proxy conforming to *atomic-view-concept* introduced in paper P0019. - end note]

using pointer =

The input type to a wrapping constructor.

static constexpr unsigned rank() const

Returns: The rank of the viewed array.

template< typename IntegralType > constexpr size_t extent(const IntegralType & r) const

Returns: The exent of dimension r when `r < rank()` and 1 when (**rank** <= r < *rank upper bound*). A default constructed view will have extent(r) == 0 for all implicit dimensions. The return value of an explicit dimension queried with a literal input value must be "constexpr" observable.

constexpr size_t size() const

Returns: The product of the extents.

using layout =

The layout type property that defaults to **void**.

using is_regular = **std::integral_constant<bool,** *B* >

Denoting by **is_regular::value** if the layout mapping is regular; *i.e.*, if there is a uniform stride between members when incrementing a particular dereferencing index and holding all other indices fixed.

template< typename IntegralType > constexpr size_t stride(const IntegralType & r) const

Requires: is_regular::value

Returns: The distance between members when index **r** is incremented by one. If is_regular::value == false the return value is undefined.

constexpr size_t span() const

Returns: A distance that is at least one plus the maximum distance between any two members of the array.

Remark: For a one-to-one layout mapping the span will equal the size.

static constexpr size_t span(size_t implicit_N0 , size_t implicit_N1 = 0 , size_t implicit_N2 = 0 , size_t implicit_N3 = 0 , size_t implicit_N4 = 0 , size_t implicit_N5 = 0 , size_t implicit_N6 = 0 , size_t implicit_N7 = 0 , size_t implicit_N8 = 0 , size_t implicit_N9 = 0)

Returns: The span of the view if it were constructed with the implicit dimensions.

constexpr pointer data() const

Returns: Pointer to the member with the minimum location.

Requires: All members are in the range `[ data() .. data() + span() )`.

reference operator()() const

Requires rank == 0.

Returns: A reference to the member of a rank zero array.

Remark: It is recommended that the requirement be enforced by conditionally defining the return type of the operator.

```
typename std::conditional< rank() == 0 , reference
                         , error_tag_invalid_access_to_non_rank_zero_view >::type
operator()() const
```

template< typename IntegralType > reference operator[](const IntegralType & i) const

Requires: rank() == 1
Requires: is_integral<IntegralType>::value
Requires: 0 <= i < extent_0()

Returns: Reference to member denoted by index **i**.

Remark: A view with a bounds-checking property should throw **std::out_of_range** when the index bounds requirement is violated.

Remark: It is recommended that the rank and type requirements be enforced by conditionally enabling the operator.

```
template< typename IntegralType >
typename std::enable_if< std::is_integral<IntegralType>::value && rank() == 1 , reference >::type
operator[]( const IntegralType & i ) const ;
```

template< typename t0 , typename t1 , ... , typename tm >
reference operator()(const t0 & i0 , const t1 & i1 , ... , const tm & im) const

Requires: std::is_integral<t#>::value
Requires: For a *proper* deference operator rank() == m + 1
Requires: For an *improper** deference operator rank() <= m
Requires: 0 <= i# < extent_#()

Returns: Reference to member associated with multi-index (i0,i1,...,im).

Remark: Index arguments are accepted as constant references of a template type to defer type promotion of these arguments until they appear in the layout mapping computation. This has been demonstrated to better enable conventional compilers to optimize code containting the layout mapping computation without the need for specialized pattern recognition of **view::operator()**.

Remark: The *improper* dereference operator is a necessary usability feature to allow functions to accept views of variable rank.

Remark: A view with a bounds-checking property should throw **std::out_of_range** when the index bounds requirement is violated. Note that for improper dereference operator extent_#() == 1 when rank() <= #.

Remark: It is recommended that the rank and type requirements be enforced by conditionally enabling the operators.

```
// Proper rank 4 member access operator
template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 >
typename std::enable_if< rank() == 4 &&
                         std::is_integral<t0>::value &&
                         std::is_integral<t1>::value &&
                         std::is_integral<t2>::value &&
                         std::is_integral<t3>::value
                       , reference >::type
 operator()( const t0 & i0
           , const t1 & i1
           , const t2 & i2
           , const t3 & i3
           ) const ;

// Improper rank 4 member access operator
template< typename t0 , typename t1 , typename t2 , typename t3 , typename t4 >
typename std::enable_if< rank() == 4 &&
                         std::is_integral<t0>::value &&
                         std::is_integral<t1>::value &&
                         std::is_integral<t2>::value &&
                         std::is_integral<t3>::value
                       , reference >::type
 operator()( const t0 & i0
           , const t1 & i1
           , const t2 & i2
           , const t3 & i3
           , const int i4
           , const int i5 = 0
           , const int i6 = 0
           , const int i7 = 0
           , const int i8 = 0
           , const int i9 = 0
           ) const ;
```

constexpr view()

Effect: Construct a *null* view with extent_#() == 0 for all implicit dimensions and data() == nullptr.

constexpr view(const view & rhs)

Effect: Construct a view of the array viewed by **rhs**.

Remark: There may be other *property* dependent effects.

view & operator = (const view & rhs)

Effect: Assigns **this** to view the array viewed by **rhs**.

Remark: There may be other *property* dependent effects.

constexpr view(view && rhs)

Effect: Construct a view of the array viewed by **rhs** and then **rhs** is *null* view.

Remark: There may be other *property* dependent effects.

view & operator = (view && rhs)

Effect: Assigns **this** to view the array viewed by **rhs** then assigns **rhs** to be a *null* view.

Remark: There may be other *property* dependent effects.

~view()

Effect: Assigns **this** to be a *null* view.

Remark: There may be other *property* dependent effects.

constexpr view(pointer ptr , size_t implicit_N0 = 0 , size_t implicit_N1 = 0 , size_t implicit_N2 = 0 , size_t implicit_N3 = 0 , size_t implicit_N4 = 0 , size_t implicit_N5 = 0 , size_t implicit_N6 = 0 , size_t implicit_N7 = 0 , size_t implicit_N8 = 0 , size_t implicit_N9 = 0);

Requires: The input **ptr** references memory `[ ptr .. ptr + S )` where S = **view::span(implicit_N0,implicit_N1,...,implicit_N9)**.

Effects: The *wrapping constructor** constructs a multidimensional array view of the given member memory such that all data members are in the span `[ ptr .. ptr + span() )`.

template< class UType , class ... UProperties > constexpr view(const view< UType , UProperties ... > & rhs)

Requires: This view type is assignable to the **rhs** view type. View assignability includes compatibility of the value type, dimensions, and properties.

Effect: Constructs a view of the array viewed by **rhs**.

```
view< int[][3] >      x(ptr,N0);
view< const int[][] > y( x ); // OK: compatible const from non-const and implicit from explicit dimension
view< int[][] >       z( y ); // Error: cannot assign non-const from const
```

template< class UType , class ... UProperties > view & operator = (const view< UType , UProperties ... > & rhs)

Requires: This view type is assignable to the **rhs** view type.

Effect: Assigns **this** to view the array viewed by **rhs**.

13 View Properties: Dimension and array_view

namespace std {
namespace experimental {
namespace view_property {

// Specify all implicit dimensions of a given rank
template< unsigned Rank >
struct implicit_dimensions ;

```
// If relaxed array dimension syntax is unavailable
template< size_t , size_t , size_t , size_t , size_t
    , size_t , size_t , size_t , size_t , size_t >
struct dimension ;

}}}

// For compatibility with declaration syntax of previous array_view proposal

namespace std {
namespace experimental {

template< typename T , unsigned Rank >
using array_view = typename view<T,view_property::implicit_dimensions<Rank> > ;

}}
```

14 Assignability of Views of Non-identical Types

It is essential that view of non-identical, compatible types be assignable. For example:

```
view< int[][3] > x( ptr , N0 );
view< const int[][] > y( x ); // valid assignment
```

The 'std::is_assignable' meta-function must be partial specialized to implement the view assignability rules regarding value type, dimensions, and properties.

```
template< class Utype , class ... Uprop
        , class Vtype , class ... Vprop >
struct is_assignable< view< Utype , Uprop ... >
                    , view< Vtype , Vprop ... > >
  : public integral_const< bool ,
      is_assignable< typename view< Utype , Uprop ... >::pointer
                   , typename view< Vtype , Vprop ... >::pointer >::value
      &&
      ( view< Utype , Uprop ... >::rank() == view< Vtype , Vprop ... >::rank() )
      &&
      (
        // Extent is either equal or impplicit.
        extent<Utype,#>::value == extent<Vtype,#>::value ||
        extent<Utype,#>::value == 0
      )
      &&
      // other possible conditions
      > {}
```

Assignability extends beyond the **cv** qualification of the view's data. For example, 1. implicitly dimensioned views are assignable from equal rank explicitly dimensioned views, 2. strided layout views with implicit dimensions are assignable from equal rank views with regular layout, or 3. a view with an access intent property, such as *random* or *restrict* may be assigned from a view without such a property.

15 Subview of View

The capability to **easily** extract subviews of a view, or subviews of subviews, is essential for usability. Non-trivial subviews of regular views will often have **view_layout_stride**.

```
using U = view< int[][][] > ;

U x(buffer,N0,N1,N2);

// Using std::pair<int,int> for an integral range
auto y = subview( x , std::pair<int,int>(1,N0-1) , std::pair<int,int>(1,N1-1) , 1 );

assert( y.rank() == 2 );
assert( y.extent(0) == N0 - 2 );
assert( y.extent(0) == N1 - 2 );
assert( & y(0,0) == & x(1,1,1) );
```

```
// Using initializer_list of size 2 as an integral range
auto z = subview( x , 1 , {1,N1-1} , 1 );

assert( z.rank() == 1 );
assert( & z(0) == & x(1,1,1) );

// Conveniently extracting subview for all of a extent
// without having to explicitly extract the dimensions.
auto x = subview( x , view_property::all , 1 , 1 );
```

Subview types are generated with a meta-function.

```
namespace std {
namespace experimental {
namespace view_property {

template< typename ViewType , class ... Indices_And_Ranges >
struct subview_type ;

struct all_type {};
constexpr all_type all = all_type();

}}}

namespace std {
namespace experimental {

template< typename ViewType , class ... Indices_And_Ranges >
using subview_t = typename view_property::subview_type< ViewType , Indices_And_Ranges >::type ;

template< typename DataType , class ... Parameters , class ... Indices_And_Ranges >
subview_t< view< DataType, Parameters ... > , Indices_And_Ranges ... >
subview( const view< DataType, Parameters ... > & , Indices_And_Ranges ... );

template< typename T >
struct is_integral_range ;

}}
```

template< typename T > struct is_integral_range : public integral_constant<bool,*F*>

Returns: Meta function indicating whether T is an integral range.

template< typename ViewType , class ... Indices_And_Ranges > struct subview_type ;

Requires: ViewType::rank() == sizeof...(Indices_And_Ranges)
Requires: Each parameter in Indices_And_Ranges is either is_integral<T> or is_integral_range<T>.

Returns: The view type of the subview from the input view and parameter pack of indices and integral ranges. The rank of the subview is equal to the number of integral ranges in the parameter pack. When a dimension of the source **ViewType** is explicit and the corresponding range argument is **all** then the dimension of the resulting view type is explicit and equal to the source dimension

16 Limited iterator interface

A **view** may have a non-isomorphic mapping between its multi-index space (domain) and span of member memory (range). For example, a subview or dimension padded view will be non-isomorphic. An iterator for the members of a non-isomorphic view must be non-trivial in order to skip over non-member spans of memory. Thus a general iterator implementation would necessarily be non-trivial both in state and algorithm. As such we provide a very limited iterator interface conforming to **24.6.5 range access** for a rank-one view with isomorphic layout (*e.g.*, default, **layout_left**, **layout_right**) and no incompatible access intent properties (*e.g.*, the **reference** type is truly a reference and not a proxy). For example, a simple **view<T[]>** will have **begin** and **end** overloads.

```
namespace std {
```

template< class T , class ...P >
typename std::enable_if< *(rank one and isomorphic layout and no incompatible access intent properties)* , typename view<T,P...>::pointer >::type
begin(const std::experimental::view<T,P...> & v)
{ return v.data(); }

template< class T , class ...P >
typename std::enable_if< *(rank one and isomorphic layout and no incompatible access intent properties)* , typename view<T,P...>::pointer >::type
end(const std::experimental::view<T,P...> & v)
{ return v.data() + v.size(); }

}

Note that in the more general case of an isomorphic view of any rank a pointer (iterator) range for view member data can be queried.

```
template< typename T , class ... P >
void foo( view<T,P...> a )
{
  if ( std::is_reference< typename view<T,P...>::reference >::value && a.size() == a.span() ) {
    // Iteration via pointer type is valid and performant
    typename view<T,P...>::pointer
      begin = a.data() ,
      end   = a.data() + a.span() ;
  }
}
```

17 View Property : Member Access Array Bounds Checking

namespace std {
namespace experimental {
namespace view_property {
struct bounds_checking ;
}}}

Array bounds checking is an invaluable tool for debugging user code. This functionality traditionally requires global injection through special compiler support. In large, long running code global array bounds checking introduces a signficant overhead that impedes the debugging process. A member access array bounds checking view property allows the selective injection of array bounds checking and removes the need for special compiler support.

```
// User enables array bounds checking for selected views.

using x_property = typename std::conditional< ENABLE_ARRAY_BOUNDS_CHECKING , view_property::bounds_checking , void >::type ;

view< int[][][3] , x_property > x(ptr,N0,N1);
```

Adding **bounds_checking** to the properties of a view has the effect of introducing an array bounds check to each member access operation. If the requirement `0 <= i# < extent_#()` fails **std::out_of_range** is thrown.